

Facultatea de Matematică și Informatică
Universitatea din București

Proiect #2

Break the Login

Atacarea și Securizarea Autentificării

Aplicația: cip_auth

Cristescu Ciprian
Facultatea de Matematică și Informatică
`cristescuciprian.cc@gmail.com`

Aprilie 2026

Cuprins

1	Introducere	3
1.1	Descrierea aplicației	3
1.2	Arhitectura aplicației	3
1.2.1	Tabele bază de date	4
1.3	Scopul proiectului	4
2	Setup mediu de lucru	5
2.1	Mediul de lucru	5
2.2	Instalare și pornire	5
2.3	Inspecție bază de date	6
2.4	Instrumente de testare	6
2.5	Structura proiectului	6
3	Implementare MVP	7
3.1	Prezentare generală	7
3.2	Înregistrare utilizator	7
3.3	Autentificare	7
3.4	Stocare parole	8
3.5	Resetare parolă	8
4	Prezentarea vulnerabilităților	9
4.1	Password Policy slab (OWASP A07:2021)	9
4.2	Stocare nesigură a parolelor (OWASP A02:2021)	10
4.3	Brute Force / Lipsa Rate Limiting (OWASP A07:2021)	10
4.4	User Enumeration (OWASP A07:2021)	10
4.5	Gestionare nesigură a sesiunilor (OWASP A02:2021)	11
4.6	Resetare parolă nesigură (OWASP A07:2021)	12
4.6.1	4.6a — Token predictibil	12
4.6.2	4.6b — Tokenuri vechi neșterse	12
4.6.3	4.6c+d — Token fără expirare, reutilizabil	12
5	Demonstrarea atacurilor (PoC)	14
5.1	Password Policy slab	14
5.2	Stocare nesigură a parolelor (MD5)	15
5.3	Brute force / lipsă rate limiting	16
5.4	User Enumeration	17
5.5	Gestionare nesigură a sesiunilor	18
5.6	Resetare parolă nesigură	19

6	Analiza impactului	22
6.1	Impact per vulnerabilitate	22
6.1.1	Password Policy slab	22
6.1.2	Stocare nesigură a parolelor (MD5)	22
6.1.3	Lipsă Rate Limiting	23
6.1.4	User Enumeration	23
6.1.5	Sesiuni nesecurizate	23
6.1.6	Resetare parolă nesigură	23
6.2	Impactul cumulat	24
7	Implementarea fix-urilor (v2)	25
7.1	Fix 4.1 — Validare politică parolă	25
7.2	Fix 4.2 — Stocare parolă cu bcrypt	26
7.3	Fix 4.3 — Rate Limiting pe login	26
7.4	Fix 4.4 — Eliminare User Enumeration	27
7.5	Fix 4.5 — Securizarea sesiunilor	27
7.6	Fix 4.6 — Token de resetare securizat	28
8	Re-test după implementarea fix-urilor	30
8.1	Re-test 4.1 — Password Policy	30
8.2	Re-test 4.2 — Stocare parolă	30
8.3	Re-test 4.3 — Brute Force	31
8.4	Re-test 4.4 — User Enumeration	32
8.5	Re-test 4.5 — Sesiune și cookie-uri	32
8.6	Re-test 4.6 — Resetare parolă	33
9	Audit & Logging	36
9.1	Prezentare generală	36
9.2	Structura audit log	36
9.3	Implementare	36
9.4	Vizualizare în dashboard	37
9.5	Interogare directă	37
9.6	Limitări ale versiunii v1	37
10	Concluzii	39
10.1	Lecții învățate	39
10.1.1	Parole și stocare	39
10.1.2	Mesaje de eroare	39
10.1.3	Token-uri de securitate	39
10.1.4	Cookie-uri de sesiune	40
10.2	Principii de securitate demonstrate	40
10.3	Concluzie generală	40

Capitolul 1

Introducere

1.1 Descrierea aplicației

Aplicația `cip_auth` reprezintă o platformă web de autentificare dezvoltată în cadrul proiectului #2 al cursului *Dezvoltarea Aplicațiilor Software Securizate*. Aplicația simulează un sistem intern al companiei fictive **AuthX**, utilizat de angajați pentru accesul la resurse sensibile.

Proiectul urmărește un ciclu complet de securitate aplicativă, cunoscut în industrie ca **Build – Hack – Fix**:

1. **Build (v1 – vulnerabil)**: construirea unui mecanism de autentificare funcțional, dar intenționat slab din punct de vedere al securității.
2. **Hack**: demonstrarea exploatării vulnerabilităților prin tehnici de *ethical hacking* (enumerare utilizatori, brute force, token prediction etc.).
3. **Fix (v2 – securizat)**: remedierea fiecărei vulnerabilități prin implementarea controalelor corecte de securitate, urmată de re-testare.

1.2 Arhitectura aplicației

Aplicația este construită ca o aplicație web monolitică cu arhitectură **MVC simplificată**:

- **Frontend**: pagini HTML generate server-side prin motorul de template-uri Jinja2, cu stilizare CSS custom.
- **Backend**: framework-ul Python **Flask** (v3.0), care gestionează rutele HTTP, logica de autentificare și sesiunile.
- **Baza de date**: **SQLite** (via modulul `sqlite3` din biblioteca standard Python), cu trei tabele: `users`, `reset_tokens` și `audit_logs`.

1.2.1 Tabele bază de date

Câmp	Tip	Descriere
id	INTEGER PK	Identificator unic
email	TEXT UNIQUE	Username / email
password	TEXT	Hash parolă (MD5 în v1)
role	TEXT	USER / ADMIN
created_at	TIMESTAMP	Data creării contului
locked	INTEGER	Cont blocat (0/1)

Tabela 1.1: Tabelul `users`

Câmp	Tip	Descriere
id	INTEGER PK	Identificator
email	TEXT	Email asociat token-ului
token	TEXT	Token de resetare (predictibil în v1)
created_at	TIMESTAMP	Momentul generării

Tabela 1.2: Tabelul `reset_tokens`

Câmp	Tip	Descriere
id	INTEGER PK	Identificator eveniment
user_id	FK	Utilizatorul care a efectuat acțiunea
action	TEXT	LOGIN / REGISTER / LOGOUT
resource	TEXT	Resursa accesată
ip_address	TEXT	Adresa IP a clientului
timestamp	TIMESTAMP	Momentul acțiunii

Tabela 1.3: Tabelul `audit_logs`

1.3 Scopul proiectului

Conform cerințelor cursului, studentul joacă un dublu rol:

- **Security Tester** – identifică și demonstrează vulnerabilitățile prin atacuri controlate, documentate cu capturi de ecran și request/response.
- **Developer** – remediază vulnerabilitățile și validează că atacurile nu mai funcționează după implementarea fix-urilor.

Accentul cade exclusiv pe fluxurile de autentificare: înregistrare, login, gestionarea sesiunilor și resetarea parolei.

Capitolul 2

Setup mediu de lucru

2.1 Mediul de lucru

Aplicația a fost dezvoltată și testată pe **Windows 10**, folosind **Visual Studio Code** ca editor și **PowerShell** ca terminal. Browserul utilizat este Google Chrome, cu **Dev-Tools** (F12) pentru inspecția cookie-urilor și rularea comenzilor JavaScript în consolă.

- **OS:** Windows 10
- **Editor:** Visual Studio Code
- **Terminal:** PowerShell
- **Python:** 3.9+, pip, SQLite (inclus în stdlib)
- **Framework:** Flask 3.0.3

Notă: Dacă testarea se efectuează într-o mașină virtuală, hostname-ul și username-ul sunt vizibile în capturile de ecran din secțiunile PoC.

2.2 Instalare și pornire

```
1 # Din directorul proiectului
2 python -m venv venv
3 .\venv\Scripts\Activate.ps1
4 pip install -r requirements.txt      # flask==3.0.3
5
6 # v1 - vulnerabila (port 5000)
7 python app.py
8
9 # v2 - securizata (port 5001, director separat)
10 cd ..\cip_auth_fixed
11 python app.py
```

Listing 2.1: Instalare și pornire în PowerShell

La prima rulare, aplicația creează automat `cip_auth.db` cu tabelele `users`, `reset_tokens` și `audit_logs`.

2.3 Inspecție bază de date

```
1 python -c "import sqlite3; conn=sqlite3.connect('cip_auth.db');  
  \  
2 print(conn.execute('SELECT id, email, password, role FROM users  
  ').fetchall())"
```

Listing 2.2: Inspectare SQLite din PowerShell

2.4 Instrumente de testare

- **curl** — request-uri HTTP din PowerShell
- **Python 3** — scripturi de atac (brute force, enumerare tokeni)
- **sqlite3 / Python sqlite3** — inspecție directă bază de date
- **Browser DevTools (F12)** — inspecție cookie-uri, consolă JS (`document.cookie`)

2.5 Structura proiectului

```
1 cip_auth/  
2 |-- app.py                # Aplicatia Flask principala (v1)  
3 |-- requirements.txt  
4 |-- cip_auth.db          # Baza de date SQLite (auto-generata)  
5 |-- static/style.css  
6 '-- templates/  
7     |-- base.html  
8     |-- login.html  
9     |-- register.html  
10    |-- dashboard.html  
11    |-- forgot_password.html  
12    '-- reset_password.html
```

Listing 2.3: Structura directoarelor

Capitolul 3

Implementare MVP

3.1 Prezentare generală

MVP-ul implementează fluxul complet de autentificare: înregistrare, autentificare, deconectare, gestionarea sesiunilor și resetarea parolei. Versiunea v1 conține vulnerabilități intenționate introduse, marcate prin comentarii în cod.

3.2 Înregistrare utilizator

```
1 # VULNERABILITATE #1: nicio verificare de complexitate sau
   lungime
2 if not email or not password:
3     return render_template("register.html", error="Campurile
      sunt obligatorii.")
4
5 conn.execute("INSERT INTO users (email, password) VALUES (?, ?)"
6               ,
7               (email, hash_password(password)))
8 conn.commit()
```

Listing 3.1: Fragment /register — fără validare parolă (v1)

Emailul este verificat pentru unicitate prin constrângerea `UNIQUE` din baza de date. Parola trece prin `hash_password()` care aplică MD5 fără salt.

3.3 Autentificare

```
1 # VULNERABILITATE #3: fara rate limiting
2 user = conn.execute("SELECT * FROM users WHERE email = ?", (
3     email,)).fetchone()
4
5 # VULNERABILITATE #4: mesaje diferite -> user enumeration
6 if not user:
7     return render_template("login.html", error="Utilizatorul nu
8         a fost gasit!")
```



```
7 if user["password"] != hash_password(password):
8     return render_template("login.html", error="Parola este
9         gresita!")
10
11 session["user_id"] = user["id"]
12 session["email"] = user["email"]
13
14 # VULNERABILITATE #5: cookie nesecurizat, citibil din JavaScript
15 response.set_cookie("user_email", email,
16                     httponly=False, secure=False, samesite=None)
```

Listing 3.2: Fragment /login — user enumeration și cookie nesecurizat (v1)

3.4 Stocare parole

```
1 # VULNERABILITATE #2: MD5 fara salt --- crackabil instant cu
2   rainbow tables
3 def hash_password(password):
4     return hashlib.md5(password.encode()).hexdigest()
```

Listing 3.3: hash_password — MD5 fără salt (v1)

MD5 este proiectat pentru viteză, nu pentru securitate. Absența salt-ului înseamnă că două conturi cu aceeași parolă au hash-uri identice, iar tabelele rainbow le recuperează în secunde.

3.5 Resetare parolă

Fluxul de resetare introduce patru vulnerabilități:

- **4.6a — Token predictibil:** generat ca `str(int(time.time()))` — valoare ghicibilă în ± 60 de încercări.
- **4.6b — Tokenuri vechi active:** cererile anterioare nu sunt șterse.
- **4.6c — Fără expirare:** token-ul rămâne valid pe termen nelimitat.
- **4.6d — Reutilizabil:** token-ul nu este șters după prima utilizare.

```
1 # VULNERABILITATE #6a: token = timestamp Unix
2 token = str(int(time.time()))
3 conn.execute("INSERT INTO reset_tokens (email, token) VALUES (?,
4     ?)",
5             (email, token))
6
7 # Lipseste: expirare, stergere dupa utilizare, invalidare tokeni
8   vechi
```

Listing 3.4: Generare token predictibil (v1)

Capitolul 4

Prezentarea vulnerabilităților

Această secțiune descrie tehnic fiecare vulnerabilitate prezentă în versiunea v1 a aplicației `cip_auth`, cu mapare pe categoriile OWASP Top 10.

4.1 Password Policy slab (OWASP A07:2021)

OWASP mapping: A07:2021 — Identification and Authentication Failures

Descriere: Aplicația nu impune nicio politică de complexitate sau lungime minimă a parolei. Singura verificare existentă este că emailul și parola nu sunt goale (pentru a preveni erori de bază de date), dar nu există nicio validare a calității parolei. Un utilizator poate crea un cont cu parola "1" sau "a".

Cod vulnerabil (din `app.py`):

```
1 email      = request.form.get("email", "").strip()
2 password   = request.form.get("password", "")
3
4 # Singura verificare: campurile sa nu fie goale
5 if not email or not password:
6     return render_template("register.html",
7                             error="Campurile sunt obligatorii.")
8
9 # VULNERABILITATE #1: Nicio validare a calitatii parolei.
10 # Parola "1" sau "a" trece fara nicio obiectie.
11 # Nu exista verificare de lungime minima, complexitate sau
12 # comparare cu liste de parole comune (rockyou.txt etc.).
```

Risc: Permite crearea de conturi cu parole extrem de slabe, vulnerabile la atacuri de tip *credential stuffing* și *dictionary attack*.

Remediere (v2): Funcție `validate_password()` — lungime minimă 8 caractere, cel puțin o literă mare, cel puțin o cifră. Validarea se aplică atât la înregistrare cât și la resetarea parolei.

4.2 Stocare nesigură a parolelor (OWASP A02:2021)

OWASP mapping: A02:2021 — Cryptographic Failures

Descriere: Parolele sunt hashuite cu algoritmul **MD5** fără utilizarea unui *salt*. MD5 este considerat complet compromis pentru stocarea parolelor, din mai multe motive:

- **Viteză:** MD5 poate calcula miliarde de hash-uri pe secundă pe hardware modern (GPU).
- **Tabele rainbow:** Există baze de date precompute cu hash-uri MD5 pentru sute de milioane de parole comune.
- **Fără salt:** Două conturi cu aceeași parolă vor avea hash-uri identice, permițând identificarea prin inspecția bazei de date.

```
1 # VULNERABILITATE #2: MD5 fara salt
2 def hash_password(password):
3     return hashlib.md5(password.encode()).hexdigest()
```

Risc: Dacă baza de date este compromisă, toate parolele pot fi recuperate în ore sau minute.

Remediere (v2): Înlocuire cu `bcrypt` (`rounds=12`) — salt unic per utilizator, intenționat lent (250ms/hash). Tabele rainbow sunt inutilizabile.

4.3 Brute Force / Lipsa Rate Limiting (OWASP A07:2021)

OWASP mapping: A07:2021 — Identification and Authentication Failures

Descriere: Endpoint-ul `/login` nu impune nicio limitare a numărului de încercări. Un atacator poate trimite mii de request-uri pe secundă fără să fie blocat, suspendat sau detectat.

Nu există:

- Limitare a ratei de request-uri (rate limiting)
- Blocare temporară a contului după N încercări eșuate
- CAPTCHA sau alte mecanisme anti-bot
- Delay artificial între încercări

Risc: Un atacator poate automatiza încercarea tuturor parolelor dintr-un dicționar (*dictionary attack*) sau poate testa sistematic toate combinațiile (*brute force*).

Remediere (v2): Rate limiter in-memory — max 5 încercări / 60 secunde per IP. Blocarea durează până la expirarea ferestrei. Tentativele eșuate sunt logate.

4.4 User Enumeration (OWASP A07:2021)

OWASP mapping: A07:2021 — Identification and Authentication Failures

Descriere: Aplicația returnează mesaje de eroare diferite în funcție de motivul eșecului autentificării:

- Email inexistent → "Utilizatorul nu a fost găsit!"
- Email existent, parolă greșită → "Parola este greșită!"

```
1 # VULNERABILITATE #4: User enumeration
2 if not user:
3     return render_template("login.html",
4                             error="Utilizatorul nu a fost gasit!"
5                             )
6 if user["password"] != hash_password(password):
7     return render_template("login.html",
8                             error="Parola este gresita!")
```

Risc: Un atacator poate determina sistematic ce adrese de email sunt înregistrate în sistem, informație utilă pentru atacuri ulterioare de brute force sau phishing.

Remediere (v2): Mesaj unic „*Credentiale invalide*” indiferent de motivul eșecului — emailul inexistent și parola greșită primesc același răspuns.

4.5 Gestionare nesigură a sesiunilor (OWASP A02:2021)

OWASP mapping: A02:2021 — Cryptographic Failures + A07:2021

Descriere: Aplicația prezintă două probleme distincte de securitate a sesiunilor, care trebuie înțelese separat.

A — Cookie demo user_email fără flaguri de securitate

La autentificare, aplicația setează un cookie suplimentar user_email intenționat ne-securizat, pentru demonstrarea vulnerabilității:

```
1 # VULNERABILITATE #5a: Cookie nesecurizat      demonstrabil via
2           DevTools/JS
3 response.set_cookie(
4     "user_email",
5     email,
6     httponly=False,    # accesibil din JavaScript (document.
7                       cookie)
8     secure=False,      # transmis si pe HTTP necriptat
9     samesite=None      # fara protectie CSRF
10 )
```

Important: Cookie-ul user_email nu este mecanismul de autentificare — accesul la dashboard se bazează pe cookie-ul de sesiune Flask (session). Vulnerabilitatea demonstrată este că user_email poate fi citit din JavaScript (document.cookie), ceea ce într-un scenariu XSS permite exfiltrarea datelor utilizatorului.

B — Secret key slab, hardcodat

```
1 # VULNERABILITATE #5b: Secret key hardcodat si predictibil
2 app.secret_key = "secret123"
```

Cookie-ul `session` al Flask este semnat (nu criptat) cu `secret_key`. Un atacator care cunoaște această valoare poate forja un cookie de sesiune valid pentru orice utilizator, obținând acces la dashboard fără a cunoaște parola.

- `httponly=False` pe `user_email`: citibil din JavaScript.
- `secure=False` pe `user_email`: transmis pe HTTP.
- `secret_key = "secret123"`: permite forjarea sesiunii Flask.

Remediere (v2): `SESSION_COOKIE_HTTPONLY=True`, `SESSION_COOKIE_SAMESITE="Strict"`, `secret_key` generat cu `secrets.token_hex(32)` (sau din variabilă de mediu), cookie `user_email` eliminat, sesiune cu expirare de 1 oră.

4.6 Resetare parolă nesigură (OWASP A07:2021)

OWASP mapping: A07:2021 — Identification and Authentication Failures

Descriere: Mecanismul de resetare a parolei conține trei vulnerabilități distincte:

4.6.1 4.6a — Token predictibil

```
1 # Token = timestamp Unix curent (ex: 1714000000)
2 token = str(int(time.time()))
```

Un atacator care știe aproximativ când a fost cerută resetarea poate enumera toate valorile posibile (intervalul de ± 60 secunde = 120 de încercări).

4.6.2 4.6b — Tokenuri vechi neșterse

La fiecare cerere de resetare, un nou token este adăugat, dar tokenurile anterioare **nu sunt șterse**. Toate rămân valide simultan.

4.6.3 4.6c+d — Token fără expirare, reutilizabil

```
1 # Nu verificam expirarea
2 reset_entry = conn.execute(
3     "SELECT * FROM reset_tokens WHERE token = ?", (token,)
4 ).fetchone()
5
6 # Tokenul NU este sters dupa utilizare
7 conn.execute("UPDATE users SET password = ? WHERE email = ?",
8             (hash_password(new_password), reset_entry["email"]))
9 conn.commit()
10 # Lipseste: DELETE FROM reset_tokens WHERE token = ?
```

Risc cumulat: Un atacator poate prelua controlul unui cont prin ghicirea sau reutilizarea unui token de resetare, fără a cunoaște parola curentă.

Remediere (v2): Token generat cu `secrets.token_urlsafe(32)` — 256 biți de entropie. Expirare după 15 minute (`expires_at` verificat în query). Ștergere din DB după prima utilizare (one-time). Tokenurile vechi șterse la fiecare nouă cerere de resetare pentru același email.

Capitolul 5

Demonstrarea atacurilor (PoC)

Această secțiune prezintă demonstrațiile practice (*Proof of Concept*) pentru fiecare vulnerabilitate identificată în versiunea v1. Toate atacurile sunt efectuate în mediu local controlat și au scop exclusiv educațional.

5.1 Password Policy slab

Aplicația nu implementează nicio validare a complexității parolei în backend. Astfel, utilizatorii pot crea conturi folosind parole extrem de slabe, precum "1", "a" sau "123".

Această vulnerabilitate permite atacatorilor să compromită conturi foarte ușor, deoarece parolele triviale pot fi ghicite rapid prin atacuri de tip brute force.

Demonstrarea atacului (PoC):

```
curl.exe -i -X POST "http://127.0.0.1:5000/register" -d "email=weak@test.com&password=1"
```

Serverul răspunde cu HTTP/1.1 302 FOUND, indicând faptul că cererea a fost acceptată și utilizatorul a fost înregistrat cu succes.

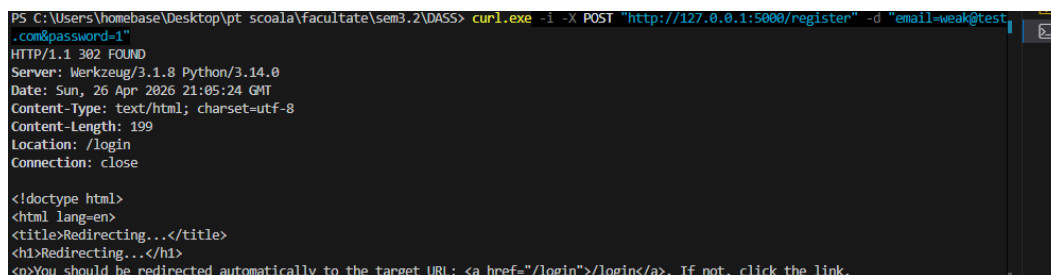


Figura 5.1: Înregistrare reușită cu o parolă trivială folosind curl

Ulterior, autentificarea cu aceleași credențiale este posibilă, demonstrând că parola extrem de slabă este acceptată și utilizată fără restricții.



Figura 5.2: Autentificare reușită cu parola trivială "1"

Impact:

Această vulnerabilitate permite utilizarea unor parole extrem de slabe, care pot fi compromise instant prin atacuri de tip brute force sau credential stuffing.

În lipsa unei politici de parole, securitatea conturilor depinde exclus de alegerea utilizatorului, ceea ce reprezintă un risc major într-un sistem real.

5.2 Stocare nesigură a parolelor (MD5)

Aplicația stochează parolele utilizatorilor folosind algoritmul MD5, fără utilizarea unui salt. MD5 este un algoritm criptografic considerat nesigur, fiind vulnerabil la atacuri de tip rainbow table și brute force.

Demonstrarea atacului (PoC):

Pentru a analiza modul de stocare al parolelor, s-a accesat direct baza de date SQLite:

```
python -c "import sqlite3; conn=sqlite3.connect('cip_auth.db');
print(conn.execute('SELECT email, password FROM users').fetchall()); conn.close()"
```

Rezultatul arată că parolele sunt stocate sub formă de hash-uri:

```
[('weak@test.com', 'c4ca4238a0b923820dcc509a6f75849b')]
```

Pentru a verifica faptul că acest hash corespunde parolei „1”, s-a calculat hash-ul MD5:

```
python -c "import hashlib; print(hashlib.md5('1'.encode()).hexdigest())"
```

Rezultatul obținut este identic:

```
c4ca4238a0b923820dcc509a6f75849b
```

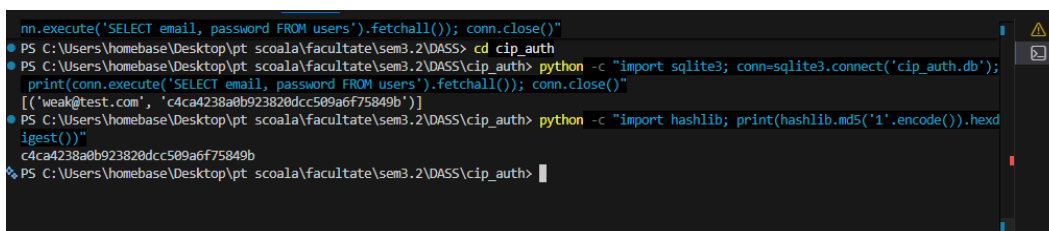


Figura 5.3: Vizualizarea parolelor în baza de date și confirmarea hash-ului MD5

Impact:

Utilizarea algoritmului MD5 fără salt face ca parolele să fie extrem de vulnerabile. Un atacator care obține acces la baza de date poate recupera rapid parolele folosind tabele pre-calculate (rainbow tables) sau atacuri brute force.

Într-un scenariu real, compromiterea bazei de date ar duce la compromiterea tuturor conturilor utilizatorilor.

5.3 Brute force / lipsă rate limiting

Aplicația nu implementează niciun mecanism de limitare a numărului de încercări de autentificare. Astfel, un atacator poate încerca un număr nelimitat de parole pentru a compromite un cont.

Demonstrarea atacului (PoC):

Pentru a demonstra această vulnerabilitate, s-a implementat un script simplu în Python care încearcă o listă de parole comune pentru un cont existent:

```
import requests

url = "http://127.0.0.1:5000/login"
email = "weak@test.com"

passwords = ["admin", "123456", "password", "test", "1", "parola"]

session = requests.Session()

for password in passwords:
    response = session.post(
        url,
        data={"email": email, "password": password},
        allow_redirects=False
    )

    print(f"Incerc parola: {password} | status={response.status_code}")

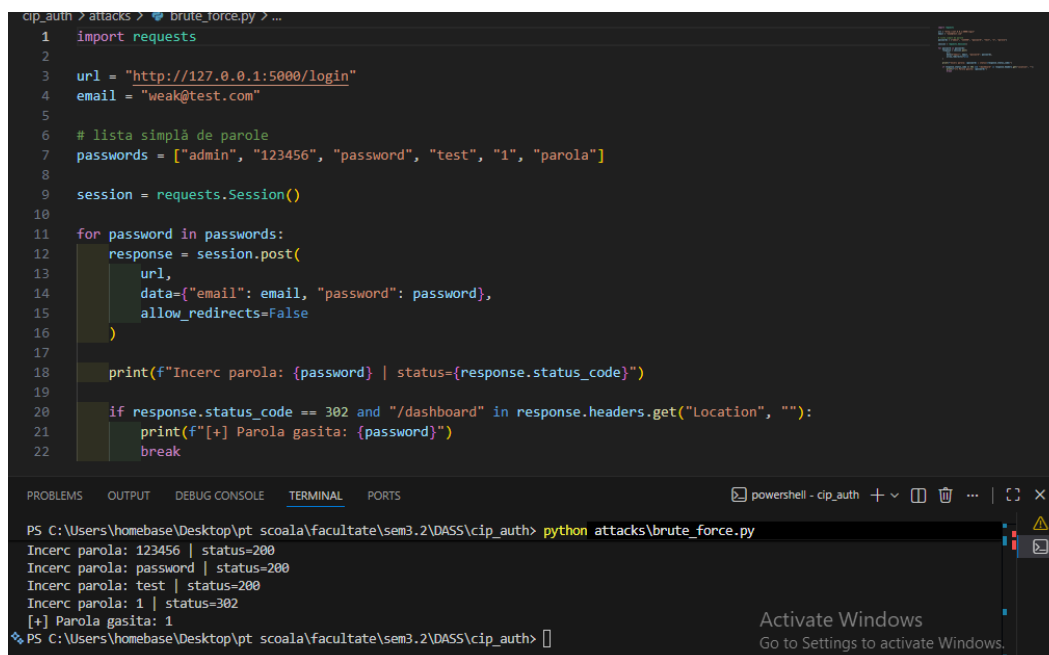
    if response.status_code == 302 and "/dashboard" in response.headers.get("Location"):
        print(f"[+] Parola gasita: {password}")
        break
```

Scriptul trimite cereri succesive către endpoint-ul de autentificare, fără a întâmpina nicio restricție sau blocare.

Rezultatul obținut este:

```
Incerc parola: admin | status=200
Incerc parola: 123456 | status=200
Incerc parola: password | status=200
Incerc parola: test | status=200
Incerc parola: 1 | status=302
[+] Parola gasita: 1
```

Codul de răspuns 302 indică autentificarea reușită (redirect către dashboard), permițând identificarea parolei corecte.



```

cip_auth > attacks > brute_force.py > ...
1  import requests
2
3  url = "http://127.0.0.1:5000/login"
4  email = "weak@test.com"
5
6  # lista simplă de parole
7  passwords = ["admin", "123456", "password", "test", "1", "parola"]
8
9  session = requests.Session()
10
11 for password in passwords:
12     response = session.post(
13         url,
14         data={"email": email, "password": password},
15         allow_redirects=False
16     )
17
18     print(f"Incerc parola: {password} | status={response.status_code}")
19
20     if response.status_code == 302 and "/dashboard" in response.headers.get("Location", ""):
21         print(f"[+] Parola gasita: {password}")
22         break

```

```

PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS\cip_auth> python attacks\brute_force.py
Incerc parola: 123456 | status=200
Incerc parola: password | status=200
Incerc parola: test | status=200
Incerc parola: 1 | status=302
[+] Parola gasita: 1
PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS\cip_auth>

```

Figura 5.4: Atac de tip brute force care identifică parola corectă

Impact:

Un atacator poate compromite conturi în mod automat prin testarea unui număr mare de parole. În lipsa unui mecanism de rate limiting sau blocare a contului, atacul poate fi executat rapid și eficient.

Această vulnerabilitate devine critică atunci când este combinată cu utilizarea unor parole slabe, permițând compromiterea conturilor în câteva secunde.

5.4 User Enumeration

Aplicația oferă mesaje de eroare diferite în funcție de cauza eșecului autentificării, permițând unui atacator să determine dacă un utilizator există sau nu în sistem.

Demonstrarea atacului (PoC):

Pentru un utilizator inexistent:

```
curl -s -X POST http://127.0.0.1:5000/login \
-d "email=nu.exista@test.com&password=test"
```

Rezultatul conține mesajul:

Utilizatorul nu a fost gasit!

Pentru un utilizator existent, dar cu parolă greșită:

```
curl -s -X POST http://127.0.0.1:5000/login \
-d "email=weak@test.com&password=gresita"
```

Rezultatul conține mesajul:

Parola este gresita!

```

PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS> curl.exe -s -X POST "http://127.0.0.1:5000/login" -d "email=nu.exista@test.com&password=test" | Select-String "Utilizatorul"

Utilizatorul nu a fost gasit!

PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS> curl.exe -s -X POST "http://127.0.0.1:5000/login" -d "email=weak@test.com&password=gresita" | Select-String "Parola"

Mesajul arata exact daca userul exista sau parola e gresita.
Parola este gresita!
<label>Parola</label>
<input type="password" name="password" placeholder="parola" required>
<a href="/forgot_password">Ai uitat parola?</a>

```

Figura 5.5: Mesaje diferite pentru utilizator inexistent și parolă greșită

Impact:

Un atacator poate identifica utilizatorii existenți în sistem pe baza mesajelor de eroare, facilitând atacuri ulterioare precum brute force sau phishing targetat.

5.5 Gestionare nesigură a sesiunilor

Aplicația setează un cookie demonstrativ numit `user_email`, fără a utiliza atribute de securitate precum `HttpOnly`, `Secure` sau `SameSite`.

În panoul **Application** -> **Cookies** din DevTools se observă că acest cookie nu are restricții de securitate, fiind configurat implicit.

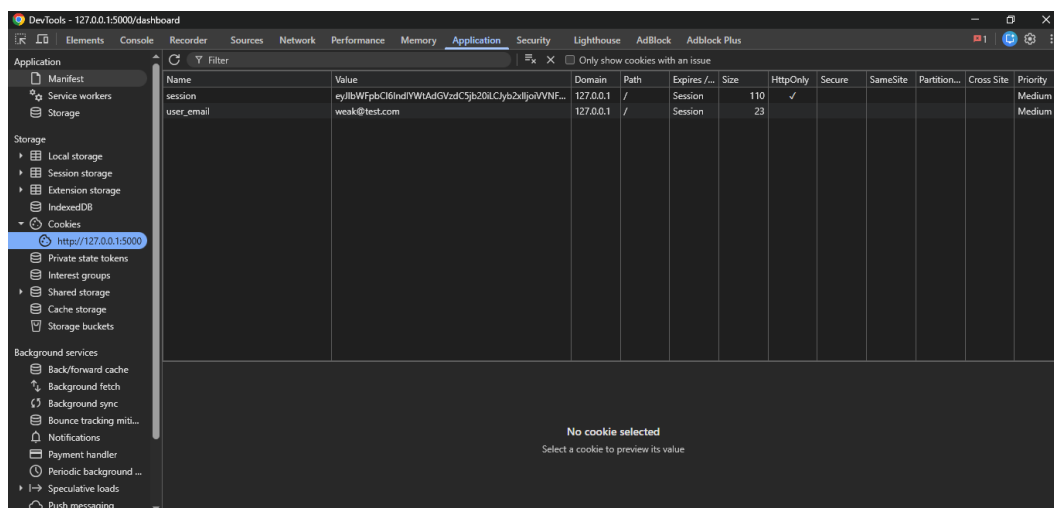


Figura 5.6: Cookie fără atribute de securitate (`HttpOnly`, `Secure`, `SameSite`)

Pentru a demonstra că acest cookie este accesibil din JavaScript, în consola browserului s-a rulat comanda:

```
document.cookie
```

Rezultatul arată că valoarea cookie-ului `user_email` este vizibilă:

`user_email=weak@test.com`

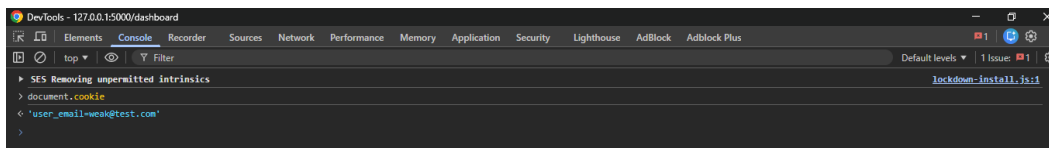


Figura 5.7: Accesarea cookie-ului din JavaScript folosind `document.cookie`

În plus, aplicația utilizează o cheie secretă hardcodată ("**secret123**") pentru gestionarea sesiunii Flask, ceea ce reprezintă o practică nesigură.

Impact:

Un cookie accesibil din JavaScript poate fi furat în cazul existenței unei vulnerabilități de tip XSS. Într-un sistem real, dacă un cookie de autentificare ar fi configurat similar, atacatorul ar putea obține sesiunea utilizatorului și ar putea prelua controlul asupra contului.

Utilizarea unei chei secrete slabe compromise, de asemenea, securitatea mecanismului de sesiune.

5.6 Resetare parolă nesigură

Funcționalitatea de resetare a parolei este implementată într-un mod nesigur, folosind token-uri predictibile bazate pe timestamp și fără mecanisme de expirare sau invalidare.

Demonstrarea atacului (PoC):

Pentru a iniția procesul de resetare, s-a accesat endpoint-ul:

`http://127.0.0.1:5000/forgot-password`

După introducerea emailului, aplicația generează un link de forma:

`http://127.0.0.1:5000/reset_password/1777241622`

Token-ul generat este un timestamp, ceea ce îl face predictibil.

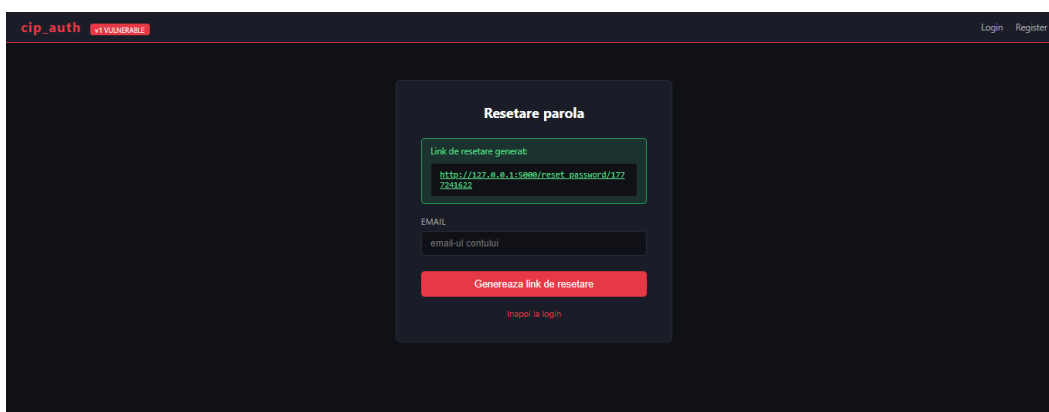
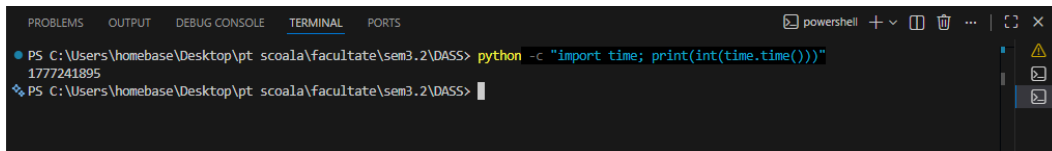


Figura 5.8: Generarea token-ului de resetare

Pentru a confirma predictibilitatea, s-a calculat timpul curent folosind:

```
python -c "import time; print(int(time.time()))"
```

Rezultatul obținut este apropiat de token-ul generat:



```
PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS> python -c "import time; print(int(time.time()))"  
1777241895  
PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS>
```

Figura 5.9: Token predictibil bazat pe timestamp

Accesând link-ul de resetare, se poate seta o nouă parolă pentru cont:

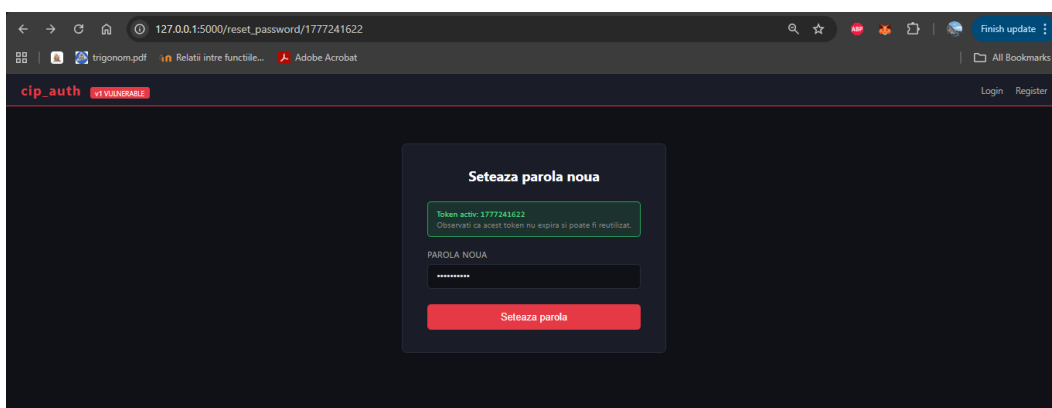


Figura 5.10: Resetarea parolei folosind token-ul

Un aspect critic este faptul că token-ul poate fi reutilizat. Același link poate fi folosit de mai multe ori pentru a reseta parola:

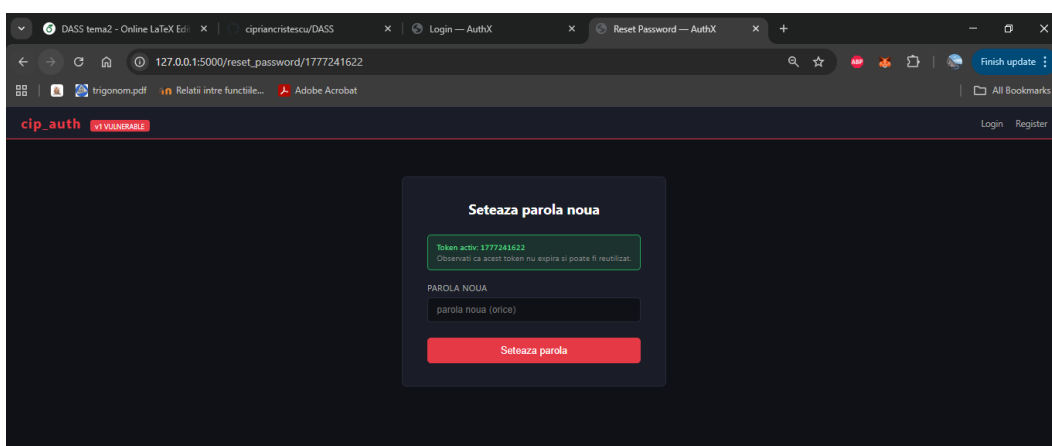


Figura 5.11: Reutilizarea token-ului de resetare

Impact:

Un atacator care obține sau ghicește token-ul poate reseta parola unui utilizator fără restricții. Deoarece token-ul nu expiră și poate fi reutilizat, accesul neautorizat poate fi menținut pe termen nelimitat.

Predictibilitatea token-ului permite chiar generarea acestuia fără a avea acces direct la aplicație, crescând semnificativ riscul de compromitere a conturilor.

Capitolul 6

Analiza impactului

Această secțiune analizează ce poate obține un atacator prin exploatarea fiecărei vulnerabilități identificate în aplicația `cip_auth v1`.

6.1 Impact per vulnerabilitate

6.1.1 Password Policy slab

Impact imediat: Utilizatorii pot crea conturi cu parole triviale ("1", "a", "123") care pot fi ghicite sau atacate cu ușurință.

Ce poate obține atacatorul:

- Acces la orice cont care folosește o parolă dintr-un dicționar comun.
- Combinat cu vulnerabilitatea de brute force (4.3), conturile cu parole slabe pot fi compromise în secunde.

Severitate: Medie (depinde de parolele alese de utilizatori).

6.1.2 Stocare nesigură a parolelor (MD5)

Impact imediat: Un atacator care obține acces la baza de date (prin SQL injection, backup neprotejat, acces fizic la server) poate recupera toate parolele în text clar.

Ce poate obține atacatorul:

- Parolele tuturor utilizatorilor în câteva minute (tabele rainbow, GPU cracking).
- Deoarece utilizatorii reutilizează parole, atacatorul poate accesa și alte servicii ale victimelor (email, internet banking, rețele sociale).
- Autentificare în aplicație cu orice cont din baza de date.

Severitate: Critică — compromiterea bazei de date înseamnă compromiterea tuturor conturilor.

6.1.3 Lipsă Rate Limiting

Impact imediat: Un atacator poate automatiza testarea a mii de parole pe secundă pentru orice cont cunoscut.

Ce poate obține atacatorul:

- Acces la orice cont al cărui email este cunoscut și a cărui parolă este prezentă într-un dicționar standard (rockyou.txt, darkweb2017-top10000).
- Prin combinare cu user enumeration (4.4), atacatorul obține atât lista de emailuri cât și capacitatea de a testa parole nelimitat.

Severitate: Înaltă — atacul este automatizabil și scalabil.

6.1.4 User Enumeration

Impact imediat: Atacatorul poate construi o listă precisă a emailurilor înregistrate în sistem.

Ce poate obține atacatorul:

- Lista de utilizatori activi, utilă pentru campanii de phishing targetate.
- Date de intrare pentru atacuri de brute force (știe pe cine să atace).
- Informații despre existența unui utilizator specific (ex: manager, admin).

Severitate: Medie (facilitator pentru atacuri mai grave).

6.1.5 Sesiuni nesecurizate

Impact imediat: Combinat cu o vulnerabilitate XSS (inclusă în aplicații reale), atacatorul poate fura cookie-ul de sesiune al unui utilizator autentificat.

Ce poate obține atacatorul:

- **Session hijacking:** accesul complet la contul victimei fără a cunoaște parola, prin simpla copiere a cookie-ului în browser-ul atacatorului.
- Pe conexiuni HTTP (fără HTTPS), cookie-ul poate fi capturat prin *man-in-the-middle* (ex: rețea WiFi publică).
- Forjarea cookie-urilor de sesiune Flask dacă secret_key este cunoscut.

Severitate: Critică în prezența XSS sau a rețelelor nesecurizate.

6.1.6 Resetare parolă nesigură

Impact imediat: Atacatorul poate prelua controlul unui cont al cărui email îl cunoaște, prin ghicirea token-ului de resetare.

Scenariu de atac:

1. Atacatorul cunoaște emailul victimei (prin user enumeration sau alte mijloace).
2. Atacatorul declanșează el însuși o cerere de resetare pentru emailul victimei.

3. Deoarece token-ul este timestamp-ul Unix, atacatorul îl poate ghici în maximum câteva zeci de încercări.
4. Atacatorul schimbă parola victimei și se autentifică cu contul ei.
5. Token-ul nu este șters după utilizare, deci poate fi reutilizat oricând.

Ce poate obține atacatorul:

- Schimbarea parolei unui utilizator cunoscut, fără a cunoaște parola curentă.
- Acces persistent la cont (token reutilizabil, fără expirare).
- **Account takeover** complet — scenariul cel mai grav în autentificare.

Severitate: Critică — duce direct la preluarea contului.

6.2 Impactul cumulat

Combinând vulnerabilitățile 4.4 și 4.6, un atacator extern (fără acces la sistem) poate prelua orice cont în câțiva pași:

1. **User enumeration (4.4):** Testează emailuri pe `/login` — mesajul diferit confirmă ce emailuri există.
2. **Declanșează resetarea (4.6a):** Trimite o cerere POST `/forgot_password` cu emailul victimei pentru a genera un token.
3. **Ghicește token-ul (4.6a):** Enumerează timestamp-urile dintr-o fereastră de câteva secunde (tipic 3–10 valori posibile).
4. **Schimbă parola (4.6d):** Trimite noua parolă la `/reset_password/<token>`.
5. **Autentificare:** Se loghează cu emailul victimei și parola nou setată.

Total pași: 5, fără acces la sistem, fără cunoașterea parolei inițiale. Combinat cu brute force (4.3), scenariul alternativ este testarea directă a parolei pe `/login` fără niciun blocaj.

Vulnerabilitate	OWASP	Severitate	Impact
4.1 Parola slabă	A07	Medie	Conturi ușor de spart
4.2 MD5 fără salt	A02	Critică	Toate parolele compromise
4.3 Brute force	A07	Înaltă	Orice cont atacabil
4.4 User enumeration	A07	Medie	Lista utilizatori
4.5 Cookie nesigur	A02+A07	Critică	Session hijacking
4.6 Reset token	A07	Critică	Account takeover

Tabela 6.1: Sumar impact vulnerabilități

Capitolul 7

Implementarea fix-urilor (v2)

Versiunea v2 a aplicației `cip_auth` este implementată în folderul `cip_auth_fixed/` și rulează pe portul **5001**. Fiecare vulnerabilitate identificată în v1 a fost remediată printr-un control specific, descris în secțiunile de mai jos.

7.1 Fix 4.1 — Validare politică parolă

Problemă: v1 accepta parole de un singur caracter ("1", "a").

Soluție: Funcție dedicată de validare, apelată la register și la reset password.

```
1 import re
2
3 def validate_password(password):
4     """Returnează mesajul de eroare sau None dacă parola e
5     validă."""
6     if len(password) < 8:
7         return "Parola trebuie să aibă cel puțin 8 caractere."
8     if not re.search(r"[A-Z]", password):
9         return "Parola trebuie să conțină cel puțin o literă
10        mare (A-Z)."
11     if not re.search(r"[0-9]", password):
12         return "Parola trebuie să conțină cel puțin o cifră
13        (0-9)."
14     return None
```

Apelul în ruta `/register`:

```
1 err = validate_password(password)
2 if err:
3     return render_template("register.html", error=err)
```

De ce e sigur: Parola "1" → respinsă (*sub 8 caractere*). Parola "password" → respinsă (*nicio cifră, nicio literă mare*). Parola "Parola1" → acceptată.

7.2 Fix 4.2 — Stocare parolă cu bcrypt

Problemă: v1 folosea MD5 fără salt — ușor de spart în câteva secunde cu tabele rainbow.

Soluție: Înlocuire completă cu bcrypt, care generează automat un salt unic și aplică un cost computațional configurabil.

```

1 import bcrypt
2
3 def hash_password(password):
4     # rounds=12      ~250ms/hash      brute force impractical
5     return bcrypt.hashpw(
6         password.encode(), bcrypt.gensalt(rounds=12)
7     ).decode()
8
9 def verify_password(password, hashed):
10    return bcrypt.checkpw(password.encode(), hashed.encode())

```

De ce e sigur:

- Fiecare parolă primește un salt unic generat criptografic — două conturi cu aceeași parolă vor avea hash-uri complet diferite.
- rounds=12 înseamnă $2^{12} = 4096$ iterații — intenționat lent. La 10^8 parole/secundă cu MD5, un GPU modern face sub 400 parole/secundă cu bcrypt round 12.
- Tabele rainbow sunt imposibile deoarece salt-ul este diferit per utilizator.

```

1 # Comparatie: MD5 vs bcrypt in baza de date
2 # v1 (MD5): c4ca4238a0b923820dcc509a6f75849b  <- crackabil
   instant
3 # v2 (bcrypt): $2b$12$eImiTXuWVxfM37uY4JANjQ...  <- imposibil de
   crack rapid

```

7.3 Fix 4.3 — Rate Limiting pe login

Problemă: v1 permitea mii de cereri pe secundă fără nicio blocare.

Soluție: Rate limiter implementat in-memory (fără dependențe externe), bazat pe o fereastră glisantă de 60 de secunde per IP.

```

1 _login_attempts = {} # { ip: [timestamp, ...] }
2 RATE_WINDOW, RATE_MAX = 60, 5
3
4 def is_rate_limited(ip):
5     cutoff = time.time() - RATE_WINDOW
6     _login_attempts[ip] = [t for t in _login_attempts.get(ip,
7         []) if t > cutoff]
8     attempts = _login_attempts.get(ip, [])
9     if len(attempts) >= RATE_MAX:

```

```

9         return True, max(int(RATE_WINDOW - (time.time() -
10             attempts[0])), 1)
11     return False, 0
12
13 # In ruta /login:
14 blocked, remaining = is_rate_limited(ip)
15 if blocked:
16     return render_template("login.html",
17                             error=f"Prea multe incercari.
18                                 Reincedea in {remaining}s.")
19
20 # dupa esec:
21 _login_attempts.setdefault(ip, []).append(time.time())
22 log_action(None, "LOGIN_FAILED", "auth", ip)
23
24 # dupa succes:
25 _login_attempts.pop(ip, None)

```

De ce e sigur: După 5 eșecuri în 60 de secunde, IP-ul este blocat pentru restul ferestrei. Scriptul de brute force din PoC 4.3 se oprește după 5 cereri. Tentativele eșuate sunt logate în `audit_logs`.

7.4 Fix 4.4 — Eliminare User Enumeration

Problemă: v1 returna mesaje diferite în funcție de motivul eșecului.

Soluție: Un singur mesaj de eroare, indiferent de motiv.

```

1 # v1 (vulnerabil):
2 if not user:
3     return render_template("login.html",
4                             error="Utilizatorul nu a fost gasit!"
5                             )
6
7 if user["password"] != hash_password(password):
8     return render_template("login.html",
9                             error="Parola este gresita!")
10
11 # v2 (fix):
12 if not user or not verify_password(password, user["password"]):
13     record_failed_attempt(ip)
14     log_action(None, "LOGIN_FAILED", "auth", ip)
15     return render_template("login.html",
16                             error="Credentiale invalide.")

```

De ce e sigur: Un atacator nu mai poate distinge între *email inexistent* și *parolă greșită* — enumerarea utilizatorilor prin mesaj de eroare devine imposibilă.

7.5 Fix 4.5 — Securizarea sesiunilor

Problemă: v1 avea `secret_key` hardcodat, cookie `user_email` fără `HttpOnly/Secure`, fără expirare sesiune.

Soluție: Configurare completă de securitate pentru sesiuni.

```

1 import os, secrets
2
3 # Secret key din variabila de mediu (nu hardcodat)
4 app.secret_key = os.environ.get("SECRET_KEY") or secrets.
    token_hex(32)
5
6 # Configurare sesiune securizata
7 app.config.update(
8     SESSION_COOKIE_HTTPONLY=True,          # nu accesibil din
        JavaScript
9     SESSION_COOKIE_SECURE=False,          # False doar local (HTTP
        ); True obligatoriu in productie (HTTPS)
10    SESSION_COOKIE_SAMESITE="Strict",      # protectie CSRF
11    PERMANENT_SESSION_LIFETIME=timedelta(hours=1),
12 )

```

Notă despre SESSION_COOKIE_SECURE: Valoarea `False` este utilizată exclusiv în mediul de dezvoltare local, unde aplicația rulează pe `http://localhost`. Flagul `Secure` instruește browser-ul să trimită cookie-ul **doar pe conexiuni HTTPS**. Într-un mediu de producție, această valoare **trebuie** setată la `True` și aplicația **trebuie** să ruleze în spatele unui server cu TLS/HTTPS configurat. Fără aceasta, cookie-ul de sesiune poate fi interceptat pe rețele nesecurizate (atac MITM).

În ruta de login:

```

1 session.permanent = True    # activeaza
    PERMANENT_SESSION_LIFETIME
2 # Nu mai setam cookie-ul user_email nesecurizat
3 return redirect(url_for("dashboard"))

```

De ce e sigur:

- `HttpOnly=True`: cookie-ul `session` nu mai poate fi citit din JavaScript — XSS nu mai poate fura sesiunea.
- `SameSite=Strict`: browser-ul nu trimite cookie-ul pe request-uri cross-site — protecție împotriva CSRF.
- `secret_key` generat aleatoriu: nu mai poate fi ghicit sau extras din cod — forjarea sesiunii devine imposibilă.
- Sesiunea expiră după 1 oră — sesiunile abandonate nu rămân valide indefinit.

7.6 Fix 4.6 — Token de resetare securizat

Problemă: v1 folosea `str(int(time.time()))` — predictibil, reutilizabil, fără expirare.

Soluție: Token criptografic, one-time, cu expirare de 15 minute.

```
1 # /forgot_password: stergem tokenurile vechi, generam token
   criptografic
2 conn.execute("DELETE FROM reset_tokens WHERE email = ?", (email
   ,))
3 token = secrets.token_urlsafe(32) # 256 biti de
   entropie
4 expires_at = (datetime.utcnow() + timedelta(minutes=15)).
   isoformat()
5 conn.execute(
6     "INSERT INTO reset_tokens (email, token, expires_at) VALUES
   (?, ?, ?)",
7     (email, token, expires_at)
8 )
9
10 # /reset_password/<token>: verificam token SI expirare simultan
11 now = datetime.utcnow().isoformat()
12 reset_entry = conn.execute(
13     "SELECT * FROM reset_tokens WHERE token = ? AND expires_at >
   ?",
14     (token, now)
15 ).fetchone()
16 if not reset_entry:
17     return render_template("reset_password.html",
18                             error="Token invalid sau expirat.",
19                             token=token)
20 # One-time use: stergem tokenul dupa utilizare
21 conn.execute("DELETE FROM reset_tokens WHERE token = ?", (token
   ,))
22 conn.commit()
```

De ce e sigur:

- `secrets.token_urlsafe(32)` generează 32 octeți aleatori — spațiu de căutare 2^{256} , imposibil de enumerat.
- Expirare după 15 minute — un token capturat ulterior nu mai este utilizabil.
- Ștergere după utilizare — același token nu poate fi folosit de două ori.
- Tokenurile vechi sunt șterse la fiecare nouă cerere de resetare.

Capitolul 8

Re-test după implementarea fix-urilor

Această secțiune repetă fiecare atac demonstrat în Capitolul 5 (PoC), aplicat de data aceasta pe versiunea v2 (`cip_auth_fixed/`, port **5001**). Scopul este să demonstreze că fix-urile implementate blochează efectiv atacurile.

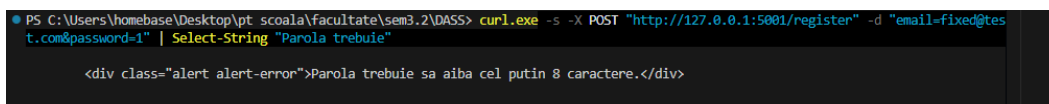
8.1 Re-test 4.1 — Password Policy

Pe versiunea securizată (v2), s-a încercat înregistrarea unui utilizator folosind o parolă extrem de slabă ("1").

```
curl.exe -s -X POST "http://127.0.0.1:5001/register" -d "email=fixed@test.com&passw
```

Aplicația nu mai acceptă parola și returnează mesajul:

Parola trebuie sa aiba cel puțin 8 caractere.



```
PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DA55> curl.exe -s -X POST "http://127.0.0.1:5001/register" -d "email=fixed@tes
t.com&password=1" | Select-String "Parola trebuie"

<div class="alert alert-error">Parola trebuie sa aiba cel puțin 8 caractere.</div>
```

Figura 8.1: Parola slabă este respinsă în versiunea v2

Concluzie: vulnerabilitatea a fost remediată. Aplicația nu mai permite crearea con-
turilor cu parole triviale.

8.2 Re-test 4.2 — Stocare parolă

În versiunea securizată (v2), s-a verificat modul în care parolele sunt stocate în baza
de date după crearea unui cont valid.

Pentru test, s-a creat un cont folosind parola "Parola12":

```
curl.exe -i -X POST "http://127.0.0.1:5001/register" -d "email=fixed@test.com&passw
```

Serverul răspunde cu HTTP 302 FOUND, indicând faptul că utilizatorul a fost înregistrat
cu succes.

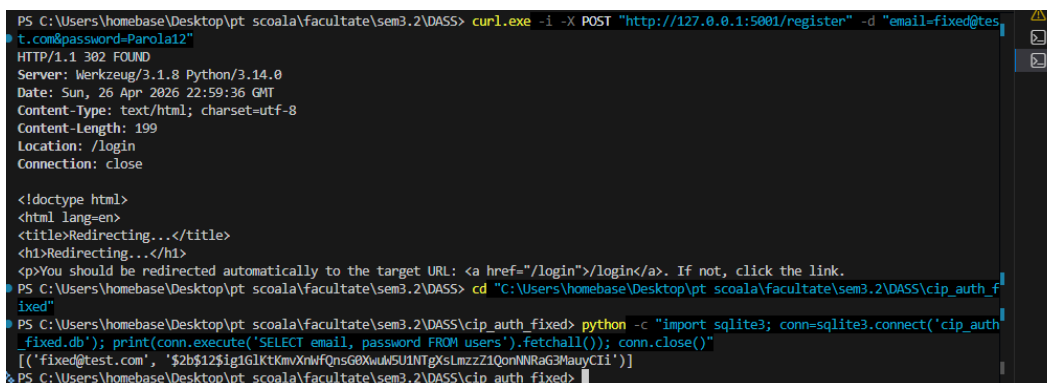
Ulterior, baza de date a fost inspectată folosind comanda:

```
python -c "import sqlite3; conn=sqlite3.connect('cip_auth_fixed.db'); print(conn.ex
```

Rezultatul obținut este:

```
[('fixed@test.com', '$2b$12$...')]
```

Se observă că parola este stocată folosind bcrypt, identificabil prin prefixul \$2b\$12\$, și nu mai este stocată în format MD5.



```
PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS> curl.exe -i -X POST "http://127.0.0.1:5001/register" -d "email=fixed@tes
t.com&password=Parola12"
HTTP/1.1 302 FOUND
Server: Werkzeug/3.1.8 Python/3.14.0
Date: Sun, 26 Apr 2026 22:59:36 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 199
Location: /login
Connection: close

<!doctype html>
<html lang=en>
<title>Redirecting...</title>
<h1>Redirecting...</h1>
<p>You should be redirected automatically to the target URL: <a href="/login">/login</a>. If not, click the link.
PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS> cd "C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS\cip_auth_f
ixed"
PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS\cip_auth_fixed> python -c "import sqlite3; conn=sqlite3.connect('cip_auth
_fixed.db'); print(conn.execute('SELECT email, password FROM users').fetchall()); conn.close()"
[('fixed@test.com', '$2b$12$1g1KtKmvXnWfQnsG0XwUWSU1NTgXsLmzzZ1QonNNRaG3MaayCfi')]
```

Figura 8.2: Parola este stocată folosind bcrypt în versiunea v2

Concluzie: vulnerabilitatea a fost remediată. Parolele nu mai sunt stocate folosind MD5, ci folosind bcrypt, care include un salt unic și un cost computațional ridicat, făcând atacurile de tip brute force sau rainbow table inefficiente.

8.3 Re-test 4.3 — Brute Force

În versiunea v2, s-a testat rezistența aplicației la atacuri de tip brute force.

S-a utilizat un script Python care încercă mai multe parole pentru același cont:

```
python attacks/brute_force_fixed.py
```

Rezultatul obținut este:

```
Incerc parola: admin | status=200
Incerc parola: 123456 | status=200
Incerc parola: password | status=200
Incerc parola: test | status=200
Incerc parola: parola | status=200
[+] Rate limiting activ - atac blocat
```

Se observă că după un număr limitat de încercări, aplicația blochează temporar autentificarea, împiedicând continuarea atacului.


```

PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS\cip_auth_fixed> python attacks\brute_force_fixed.py
Incerca parola: admin | status=200
Incerca parola: 123456 | status=200
Incerca parola: password | status=200
Incerca parola: test | status=200
Incerca parola: parola | status=200
Incerca parola: Parola12 | status=200
[+] Rate limiting activ - atac blocat
PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS\cip_auth_fixed>

```

Figura 8.3: Atacul de tip brute force este blocat în versiunea v2

Concluzie: vulnerabilitatea a fost remediată. Implementarea rate limiting limitează numărul de încercări de autentificare și face atacurile brute force ineficiente.

8.4 Re-test 4.4 — User Enumeration

În versiunea v2, s-a verificat dacă aplicația mai oferă mesaje diferite pentru email inexistent și pentru email existent cu parolă greșită.

```
curl.exe -s -X POST "http://127.0.0.1:5001/login" -d "email=nu.exista@test.com&password=test"
```

```
curl.exe -s -X POST "http://127.0.0.1:5001/login" -d "email=fixed@test.com&password=gresita"
```

În ambele cazuri, aplicația returnează același mesaj generic:

Credentiale invalide.

```

PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS> curl.exe -s -X POST "http://127.0.0.1:5001/login" -d "email=nu.exista@test.com&password=test" | Select-String "Credentiale"
<div class="alert alert-error">Credentiale invalide.</div>

PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS> curl.exe -s -X POST "http://127.0.0.1:5001/login" -d "email=fixed@test.com&password=gresita" | Select-String "Credentiale"
<div class="alert alert-error">Credentiale invalide.</div>

PS C:\Users\homebase\Desktop\pt_scoala\facultate\sem3.2\DASS>

```

Figura 8.4: Același mesaj pentru email inexistent și parolă greșită

Concluzie: vulnerabilitatea a fost remediată. Atacatorul nu mai poate determina dacă un email există în sistem pe baza mesajelor de eroare.

8.5 Re-test 4.5 — Sesiune și cookie-uri

În versiunea v2, s-a verificat modul în care sunt gestionate cookie-urile și dacă acestea sunt protejate împotriva accesului din JavaScript.

După autentificare, în consola browserului s-a rulat:

```
document.cookie
```

Rezultatul obținut este:

```
""
```

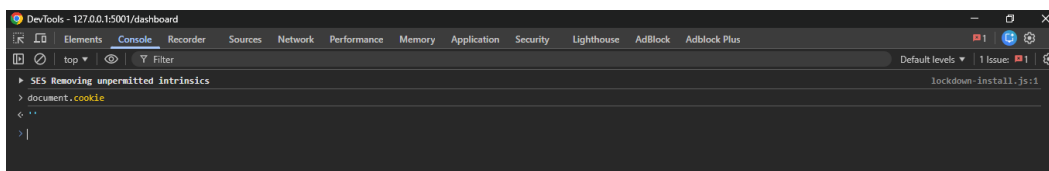


Figura 8.5: Cookie-ul de sesiune nu este accesibil din JavaScript

Se observă că nu este returnat niciun cookie relevant, ceea ce indică faptul că acesta este marcat ca `HttpOnly` și nu poate fi accesat din JavaScript.

În DevTools (Application → Cookies), se observă că:

- există doar cookie-ul `session`
- acesta are atributul `HttpOnly` activ
- atributul `SameSite` este setat la `Strict`
- cookie-ul nesecurizat `user_email` nu mai există

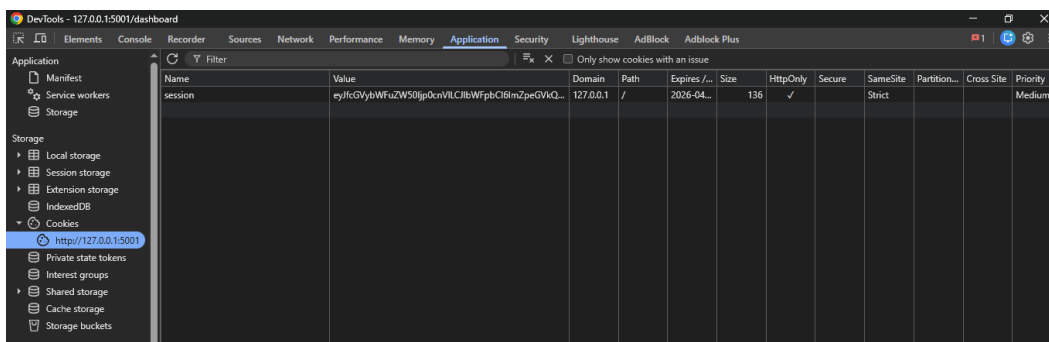


Figura 8.6: Cookie-ul de sesiune este securizat (`HttpOnly`, `SameSite=Strict`)

Concluzie: vulnerabilitatea a fost remediată. Cookie-ul de sesiune nu mai poate fi accesat din JavaScript, iar informațiile sensibile nu mai sunt expuse prin cookie-uri nesecurizate.

8.6 Re-test 4.6 — Resetare parolă

În versiunea v2, s-a verificat dacă token-ul de resetare mai este predictibil, reutilizabil sau valabil pe termen nelimitat.

După trimiterea formularului de resetare pentru utilizatorul `fixed@test.com`, aplicația generează un token lung și aleatoriu:

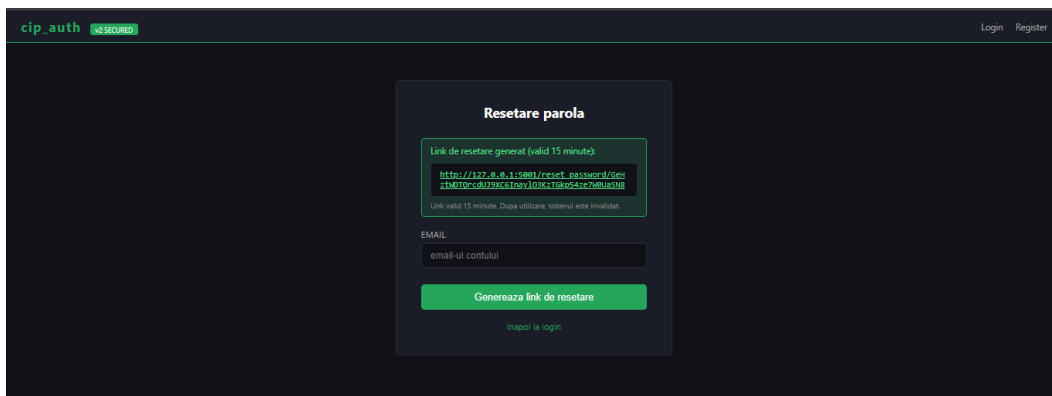


Figura 8.7: Token de resetare lung și aleatoriu în versiunea v2

Token-ul nu mai este un timestamp, ci este generat folosind `secrets.token_urlsafe(32)`, ceea ce îl face impredictibil.

Accesând linkul de resetare, parola poate fi schimbată o singură dată.

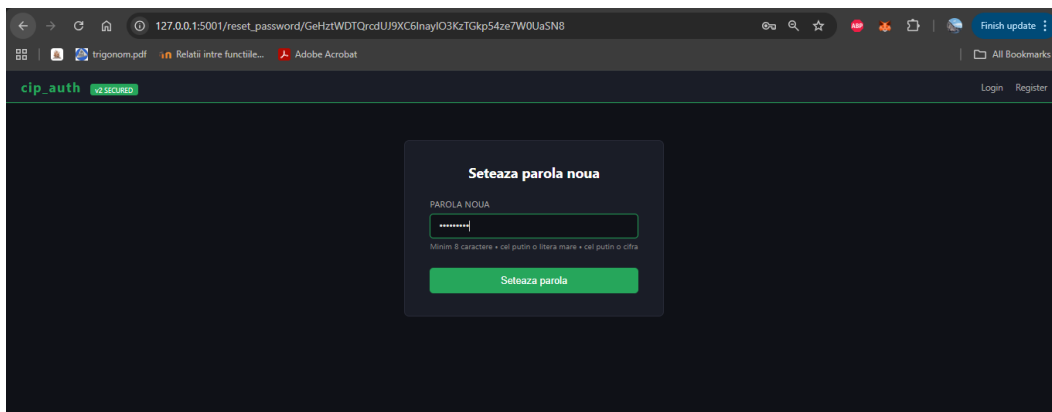


Figura 8.8: Resetarea parolei cu un token valid

După prima utilizare, același token este invalidat. Reutilizarea linkului afișează mesajul `Token invalid sau expirat`.

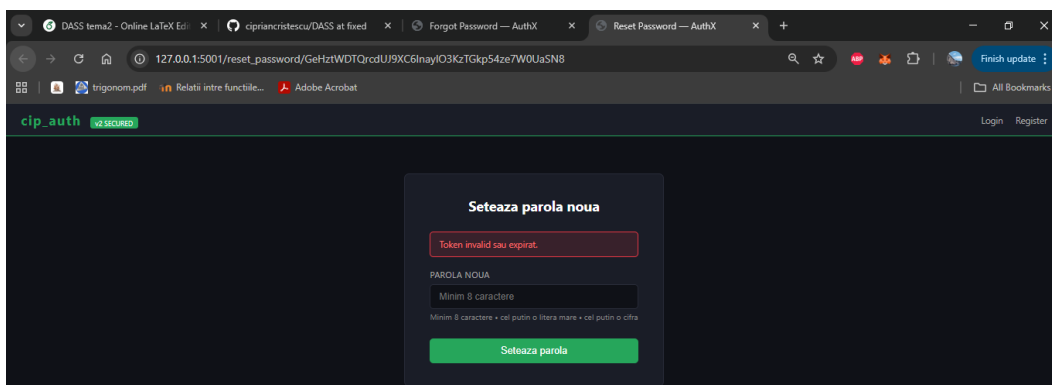


Figura 8.9: Reutilizarea token-ului este blocată în versiunea v2

Concluzie: vulnerabilitatea a fost remediată. Token-ul de resetare este aleatoriu, are expirare și este invalidat după prima utilizare.

Capitolul 9

Audit & Logging

9.1 Prezentare generală

Aplicația `cip_auth` implementează un sistem de audit logging prin tabelul `audit_logs`. Scopul este de a asigura trasabilitatea acțiunilor utilizatorilor și de a permite detectarea activității suspecte.

9.2 Structura audit log

Câmp	Tip	Descriere
<code>id</code>	INTEGER PK	Identificator unic al evenimentului
<code>user_id</code>	INTEGER FK	Utilizatorul care a efectuat acțiunea
<code>action</code>	TEXT	Tipul acțiunii (LOGIN, LOGOUT, REGISTER)
<code>resource</code>	TEXT	Resursa accesată (auth, ticket etc.)
<code>ip_address</code>	TEXT	Adresa IP a clientului
<code>timestamp</code>	TIMESTAMP	Momentul exact al acțiunii

Tabela 9.1: Schema tabelului `audit_logs`

9.3 Implementare

```
1 def log_action(user_id, action, resource, ip):
2     conn = get_db()
3     conn.execute(
4         "INSERT INTO audit_logs "
5         "(user_id, action, resource, ip_address) VALUES (?, ?,
6         ?, ?)",
7         (user_id, action, resource, ip)
8     )
9     conn.commit()
10    conn.close()
```

Funcția este apelată la fiecare acțiune relevantă:

```
1 # La inregistrare
2 log_action(None, "REGISTER", "auth", request.remote_addr)
3
4 # La login reusit
5 log_action(user["id"], "LOGIN", "auth", request.remote_addr)
6
7 # La logout
8 log_action(user_id, "LOGOUT", "auth", request.remote_addr)
```

9.4 Vizualizare în dashboard

Utilizatorul autentificat poate vizualiza ultimele 10 evenimente ale contului său în secțiunea **Jurnal de activitate** din dashboard:

```
1 logs = conn.execute(
2     "SELECT * FROM audit_logs WHERE user_id = ? "
3     "ORDER BY timestamp DESC LIMIT 10",
4     (session["user_id"],)
5 ).fetchall()
```

9.5 Interogare directă

Administratorii pot consulta log-urile direct în baza de date:

```
1 sqlite3 cip_auth.db
2
3 # Toate actiunile inregistrate
4 sqlite> SELECT * FROM audit_logs ORDER BY timestamp DESC;
5
6 # Login-uri reusite recente (v1 nu inregistreaza cele esuate)
7 sqlite> SELECT user_id, ip_address, timestamp
8         FROM audit_logs WHERE action='LOGIN'
9         ORDER BY timestamp DESC LIMIT 20;
10
11 # Inregistrari pe utilizator
12 sqlite> SELECT action, COUNT(*) as total
13         FROM audit_logs GROUP BY action;
```

Limitare importantă: Interogările de mai sus reflectă **doar acțiunile reușite**. Deoarece v1 nu loghează login-urile eșuate, un atac de brute force nu lasă nicio urmă în audit_logs.

9.6 Limitări ale versiunii v1

Implementarea actuală a audit logging-ului este funcțională, dar are câteva limitări care vor fi remediate în v2:

- Login-urile eșuate nu sunt înregistrate (se loghează doar cele reușite).
- Nu există alertare automată pentru activitate suspectă.
- Log-urile nu sunt separate de baza de date principală (risc de manipulare).

În v2, logarea tentativelor eșuate este esențială pentru detectarea atacurilor de brute force.

Capitolul 10

Concluzii

10.1 Lecții învățate

Proiectul `cip_auth` a demonstrat în mod practic că un mecanism de autentificare aparent funcțional poate fi compromis complet prin vulnerabilități fundamentale de implementare.

10.1.1 Parole și stocare

Cea mai importantă lecție în gestionarea parolelor este că **viteza este dușmanul securității**. Algoritmii rapizi precum MD5 sau SHA1 sunt proiectați pentru performanță, nu pentru securitate. Algoritmii dedicați stocării parolelor (`bcrypt`, `Argon2`, `scrypt`) introduc intenționat un cost computațional ridicat, care face atacurile brute force impractice.

O parolă `bcrypt` cu work factor 12 necesită ~250ms pentru verificare. La 100 de milioane de parole/secundă (GPU modern pentru MD5), un atacator ar testa aceeași listă în 4 parole/secundă cu `bcrypt` — o diferență de 25 de milioane de ori.

10.1.2 Mesaje de eroare

Mesajele de eroare diferențiate par o îmbunătățire a experienței utilizatorului, dar în realitate oferă informații valoroase atacatorilor. Regula de aur: **niciodată nu confirmați sau infirmați existența unui email** — returnați întotdeauna mesajul generic „Credentiale invalide”.

10.1.3 Token-uri de securitate

Orice token de securitate (resetare parolă, confirmare email, API key) trebuie:

1. Generat criptografic (`secrets.token_urlsafely()`, nu `random` sau `timestamp`).
2. Să aibă o expirare scurtă (15-60 minute pentru resetare parolă).
3. Să fie *one-time use* (șters din baza de date după prima utilizare).

10.1.4 Cookie-uri de sesiune

Flagurile `HttpOnly`, `Secure` și `SameSite` nu sunt opționale — sunt linia de apărare împotriva XSS și CSRF. Absența lor transformă orice vulnerabilitate XSS minoră într-un *session hijacking* complet.

10.2 Principii de securitate demonstrate

- **Defense in Depth:** nicio vulnerabilitate nu există în izolare — combinarea lor amplifică impactul exponențial.
- **Fail Secure:** în caz de eroare sau îndoială, sistemul trebuie să refuze accesul, nu să-l permită.
- **Least Privilege:** utilizatorii trebuie să aibă acces minim necesar.
- **Secure by Default:** configurațiile implicite trebuie să fie sigure (cookie `HttpOnly` activat implicit, rate limiting activ implicit).

10.3 Concluzie generală

Construirea versiunii vulnerabile și atacarea sa sistematică a oferit o înțelegere profundă a motivului pentru care fiecare control de securitate există. Este mult mai ușor să înțelegi de ce *bcrypt* este necesar după ce ai văzut cum MD5 este spart în secunde, sau de ce rate limiting-ul contează după ce ai rulat un script de brute force care găsește o parolă în câteva secunde.